

Lesson 02 Demo 01

Negotiating Response Format for Multi-Agent Communication

Objective: To demonstrate how multi-agent systems maintain reliable communication across mismatched data formats by declaring capabilities, negotiating a shared format at runtime, and automatically converting payloads (for example, XML ↔ JSON) when needed, while failing safely with clear, auditable logs when no common format exists

Tools required: None

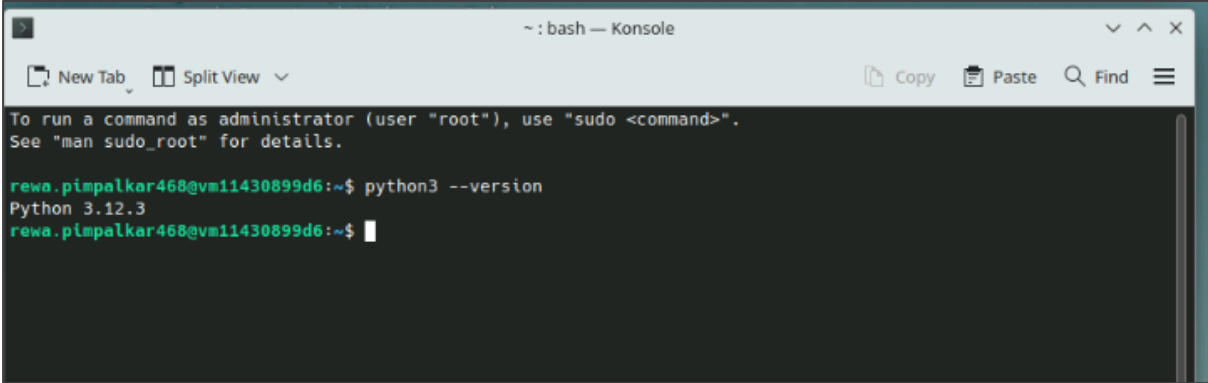
Steps to be followed:

1. Set up the format negotiation code
2. Try scenario variations in the format negotiation code

Step 1: Set up the format negotiation code

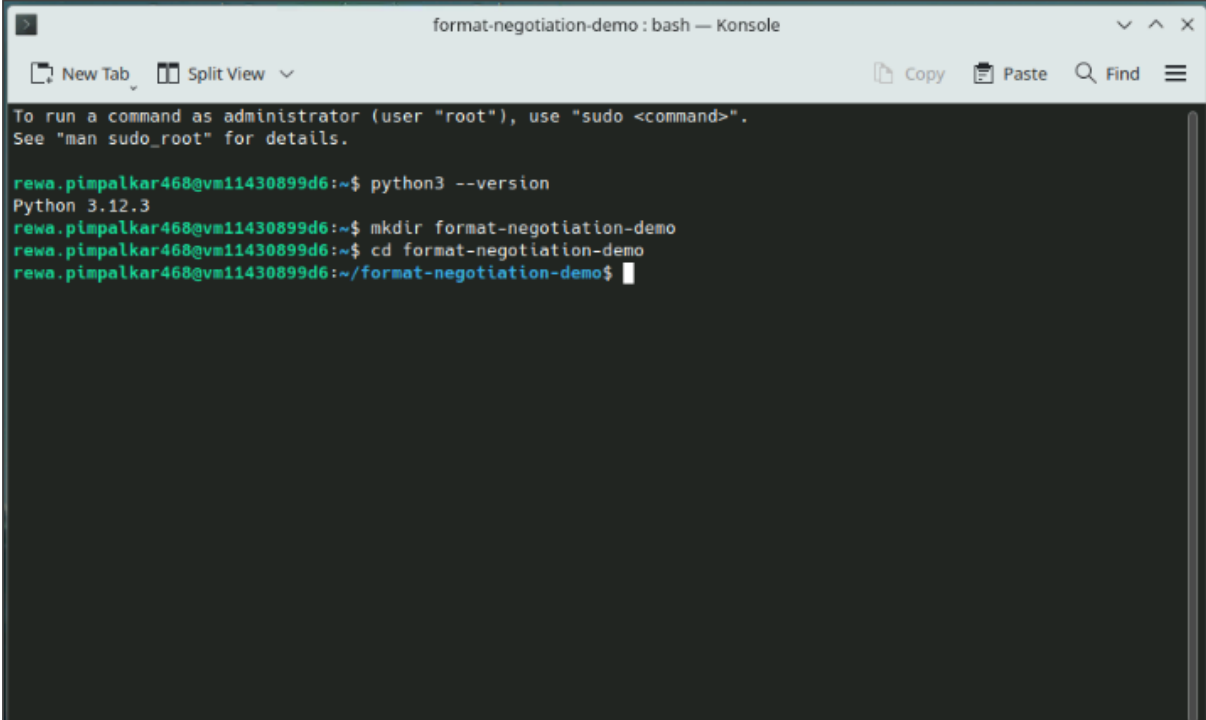
- 1.1 Open the command prompt and check if Python is installed using the following command:

python3 --version

A screenshot of a terminal window titled '~: bash — Konsole'. The terminal shows a prompt 'rewa.pimpalkar468@vm11430899d6:~\$' followed by the command 'python3 --version'. The output is 'Python 3.12.3'. The prompt is followed by a cursor. The terminal window has a menu bar with 'New Tab', 'Split View', 'Copy', 'Paste', 'Find', and a hamburger menu icon.

- 1.2 Create a new directory **format-negotiation-demo** and change into the newly created directory using the following commands:

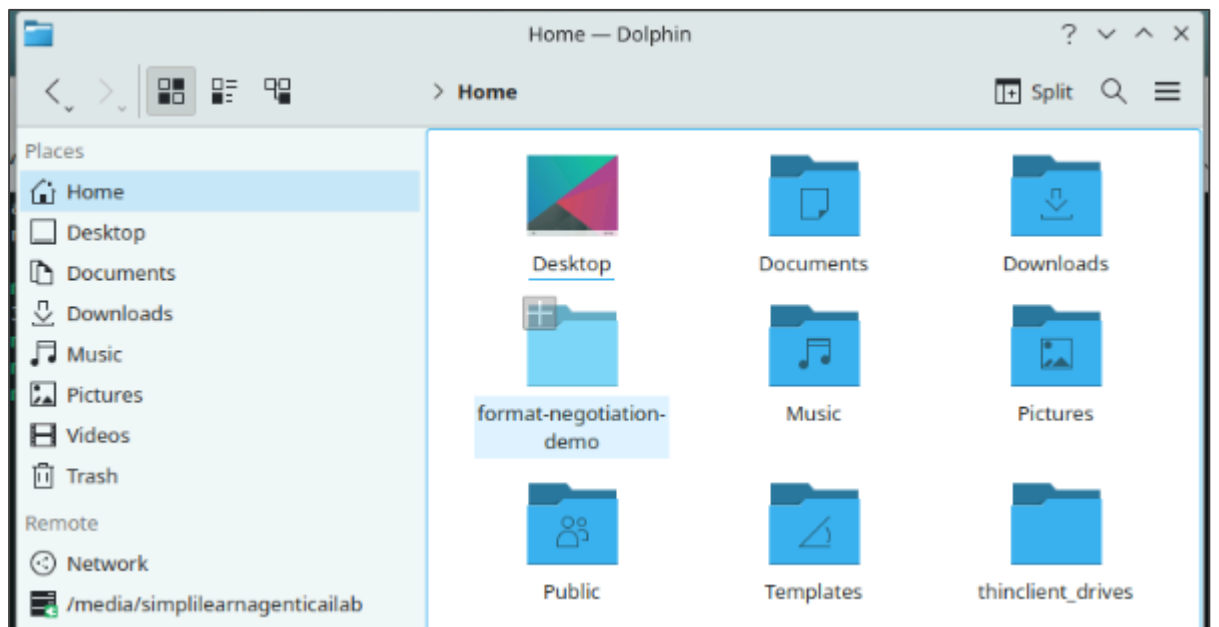
```
mkdir format-negotiation-demo
cd format-negotiation-demo
```



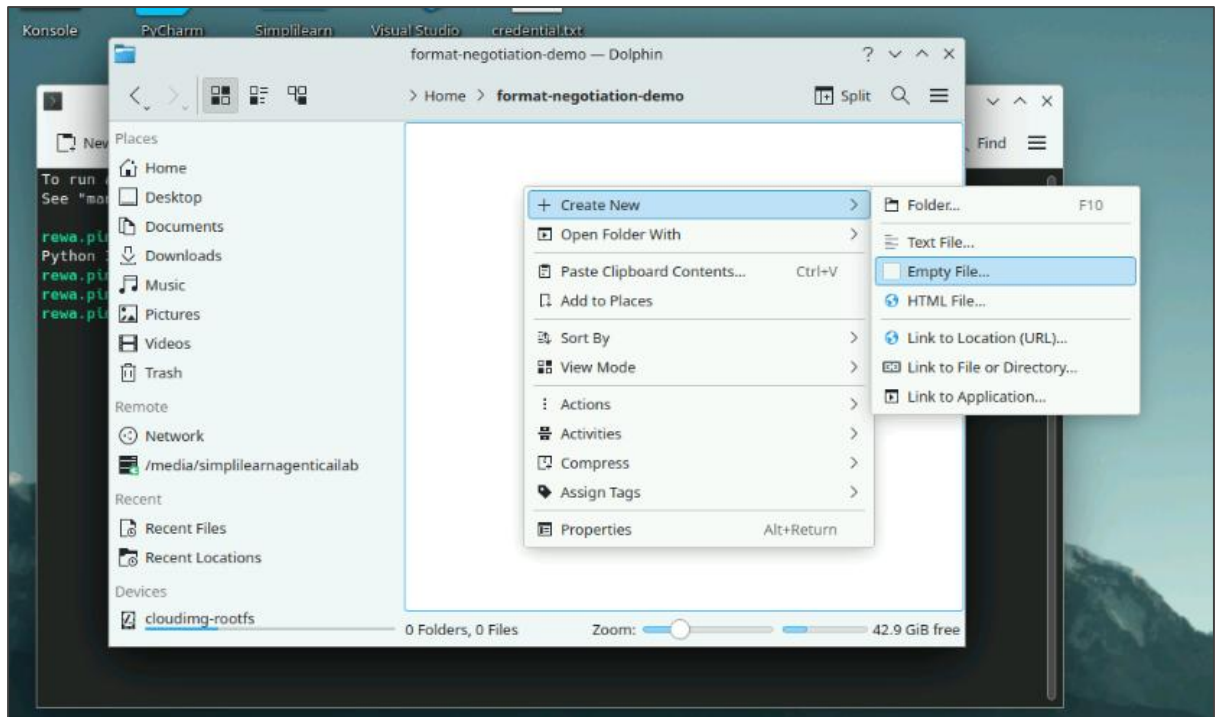
```
format-negotiation-demo : bash — Konsole
New Tab Split View
Copy Paste Find
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

rewa.pimpalkar468@vm11430899d6:~$ python3 --version
Python 3.12.3
rewa.pimpalkar468@vm11430899d6:~$ mkdir format-negotiation-demo
rewa.pimpalkar468@vm11430899d6:~$ cd format-negotiation-demo
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$
```

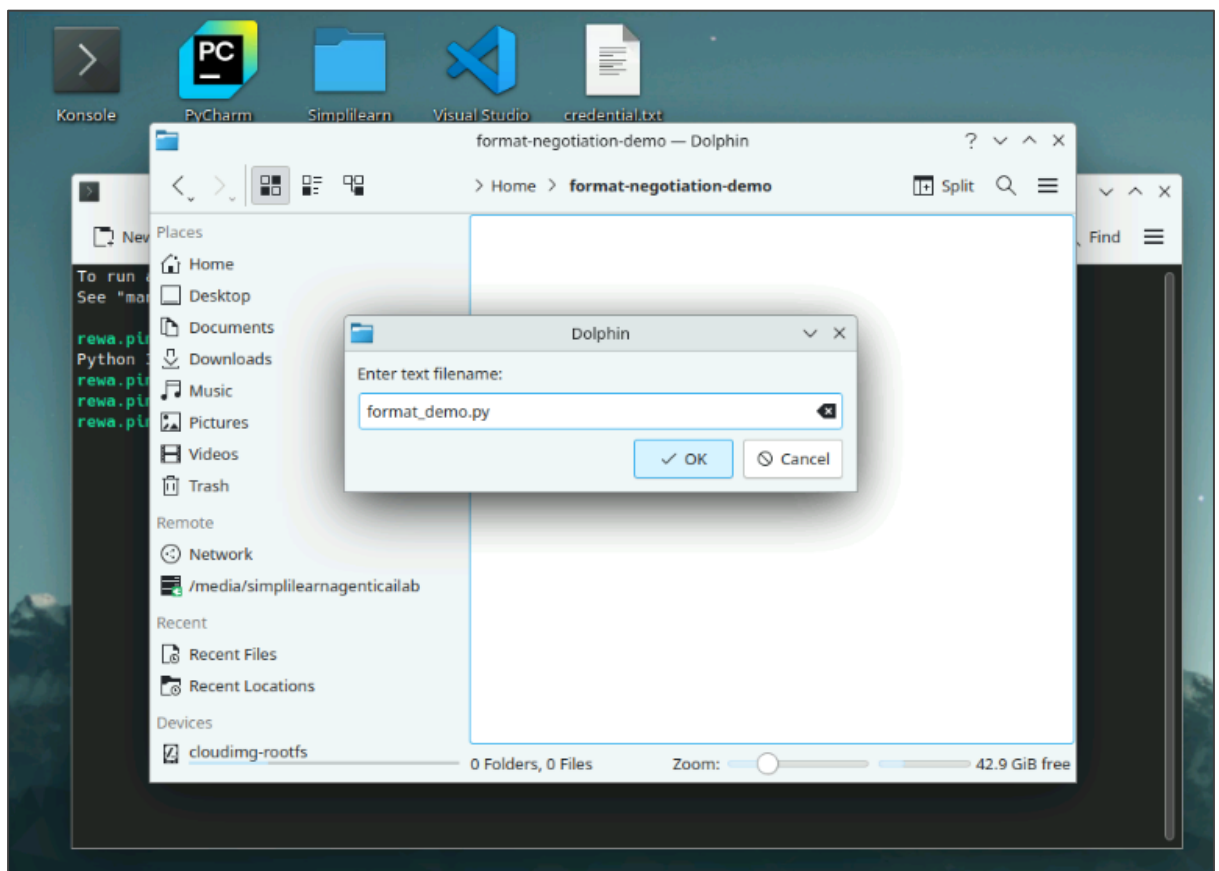
1.3 Locate and open the **format-negotiation-demo** folder



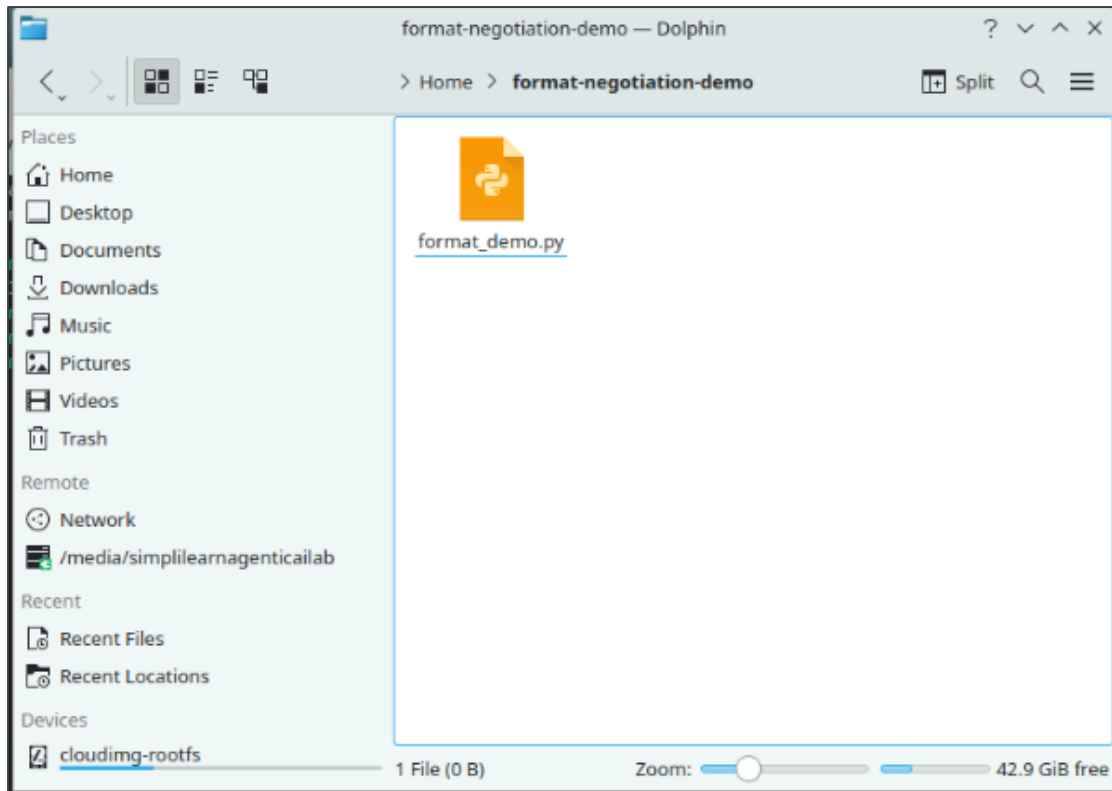
1.4 Right-click and create a new file using the option **Create New > Empty File**



1.5 Name the file **format_demo.py** and then click **OK**



The **format_demo.py** file is created.



1.6 Open **format_demo.py**, paste the following code into this file, and save it:

```
# format_demo.py — PM-friendly, single-file demo (no external packages)
# Shows the four steps: NEGOTIATE → (optional) CONVERT → DELIVER (or FALLBACK)
import json, xml.etree.ElementTree as ET
from typing import List, Dict, Any, Optional

# --- Step 3 helpers: JSON ↔ XML converters (minimal, demo-grade) ---
def dict_to_json_bytes(d: Dict) -> bytes:
    return json.dumps(d, separators=(",", ":"), ensure_ascii=False).encode("utf-8")

def json_bytes_to_dict(b: bytes) -> Dict:
    return json.loads(b.decode("utf-8"))

def dict_to_xml_bytes(d: Dict) -> bytes:
    root = ET.Element("message")
    for k, v in d.items():
        child = ET.SubElement(root, str(k))
        child.text = str(v)
    return ET.tostring(root, encoding="utf-8")

def xml_bytes_to_dict(b: bytes) -> Dict:
    root = ET.fromstring(b.decode("utf-8"))
    out = {}
    for child in root:
```

```

    text = child.text or ""
    try:
        num = float(text)
        out[child.tag] = int(num) if num.is_integer() else num
    except:
        out[child.tag] = text
    return out

def convert_bytes(src_fmt: str, dst_fmt: str, payload: bytes) -> bytes:
    src_fmt, dst_fmt = src_fmt.upper(), dst_fmt.upper()
    if src_fmt == dst_fmt:
        return payload
    # Decode to dict
    if src_fmt == "JSON":
        d = json_bytes_to_dict(payload)
    elif src_fmt == "XML":
        d = xml_bytes_to_dict(payload)
    else:
        raise ValueError("Unsupported source format")
    # Encode to destination
    if dst_fmt == "JSON":
        return dict_to_json_bytes(d)
    elif dst_fmt == "XML":
        return dict_to_xml_bytes(d)
    else:
        raise ValueError("Unsupported destination format")

# --- Step 2: Negotiation (find overlap, respect preference order) ---
def negotiate(sender_supported: List[str], receiver_supported: List[str],
              sender_pref: List[str], receiver_pref: List[str]) -> Optional[str]:
    s = {x.upper() for x in sender_supported}
    r = {x.upper() for x in receiver_supported}
    overlap = s & r
    if not overlap:
        return None
    # Try sender preference first, then receiver
    for p in [x.upper() for x in sender_pref]:
        if p in overlap:
            return p
    for p in [x.upper() for x in receiver_pref]:
        if p in overlap:
            return p
    return next(iter(overlap))

# --- Minimal Agent model (kept simple for PMs) ---
class Agent:
    def __init__(self, name: str, supported: List[str], pref: List[str]):

```

```

self.name = name
self.supported = supported
self.pref = pref

def encode(self, fmt: str, message: Dict[str, Any]) -> bytes:
    if fmt.upper() == "JSON":
        return dict_to_json_bytes(message)
    elif fmt.upper() == "XML":
        return dict_to_xml_bytes(message)
    else:
        raise ValueError("Unsupported encode format")

def receive(self, payload: bytes, fmt: str) -> bool:
    return fmt.upper() in [x.upper() for x in self.supported]

def send(self, receiver: "Agent", message: Dict[str, Any], src_format: str) -> Dict[str, Any]:
    # STEP 2: NEGOTIATE
    agreed = negotiate(self.supported, receiver.supported, self.pref, receiver.pref)
    print(f"NEGOTIATE: sender={self.name}, receiver={receiver.name}, "
          f"sender_supported={self.supported}, "
          receiver_supported={receiver.supported}, agreed={agreed}")
    if not agreed:
        # STEP 4 (fallback path when no overlap)
        print("FALLBACK: No common format. Action=abort_and_log")
        return {"ok": False, "reason": "no_common_format"}

    # Prepare "before" payload in the sender's chosen format
    payload_before = self.encode(src_format, message)

    # STEP 3: CONVERT if needed
    needs_conv = src_format.upper() != agreed.upper()
    if needs_conv:
        print(f"CONVERT: {src_format.upper()} -> {agreed.upper()}")
        payload_after = convert_bytes(src_format, agreed, payload_before)
    else:
        payload_after = payload_before

    # (Nice to show) Print payload before/after so PMs can see the wrapper
    change
    try:
        print(f"PAYLOAD BEFORE ({src_format.upper()}):")
        print(payload_before.decode("utf-8"))
        print(f"PAYLOAD AFTER ({agreed.upper()}):")
        print(payload_after.decode("utf-8"))
    except Exception:
        pass # keep demo robust even if decoding fails

```

```

# STEP 4: DELIVER
content_type = "application/json" if agreed.upper() == "JSON" else
"application/xml"
ok = receiver.receive(payload_after, agreed)
print(f"DELIVER: content_type={content_type}, ok={ok}")
return {"ok": ok, "content_type": content_type, "agreed_format": agreed}

# --- STEP 1: Define Supported Formats per Agent (metadata) ---
A = Agent("A", ["XML", "JSON"], ["XML", "JSON"]) # Sender supports XML+JSON,
prefers XML
B = Agent("B", ["JSON"], ["JSON"]) # Receiver supports JSON only
message = {"id": 101, "title": "Quarterly Report", "amount": 123.45}

# --- STEP 4: Test Interoperability (run both scenarios when executed) ---
if __name__ == "__main__":
    print("=== Success path (mismatch auto-heals) ===")
    result1 = A.send(B, message, src_format="XML") # A sends XML, agreed will be
    JSON → convert, then deliver
    print("RESULT:", result1, "\n")

    print("=== Fallback path (no overlap) ===")
    C = Agent("C", ["XML"], ["XML"]) # only XML
    D = Agent("D", ["JSON"], ["JSON"]) # only JSON
    result2 = C.send(D, message, src_format="XML")
    print("RESULT:", result2)

```

```

format_demo.py - Kate
File Edit Selection View Go Projects LSP Client Sessions Tools Settings Help
New Open Save Save As Undo Redo
format_demo.py
# format_demo.py - PT-friendly, single-file demo (no external packages)
# Shows the four steps: NEGOTIATE -> (optional) CONVERT -> DELIVER (or FALLBACK)
import json, xml.etree.ElementTree as ET
from typing import List, Dict, Any, Optional

# --- Step 3 helpers: JSON -> XML converters (minimal, demo-grade) ---
def dict_to_json_bytes(d: Dict) -> bytes:
    return json.dumps(d, separators=(",", ":"), ensure_ascii=False).encode("utf-8")

def json_bytes_to_dict(b: bytes) -> Dict:
    return json.loads(b.decode("utf-8"))

def dict_to_xml_bytes(d: Dict) -> bytes:
    root = ET.Element("message")
    for k, v in d.items():
        child = ET.SubElement(root, str(k))
        child.text = str(v)
    return ET.tostring(root, encoding="utf-8")

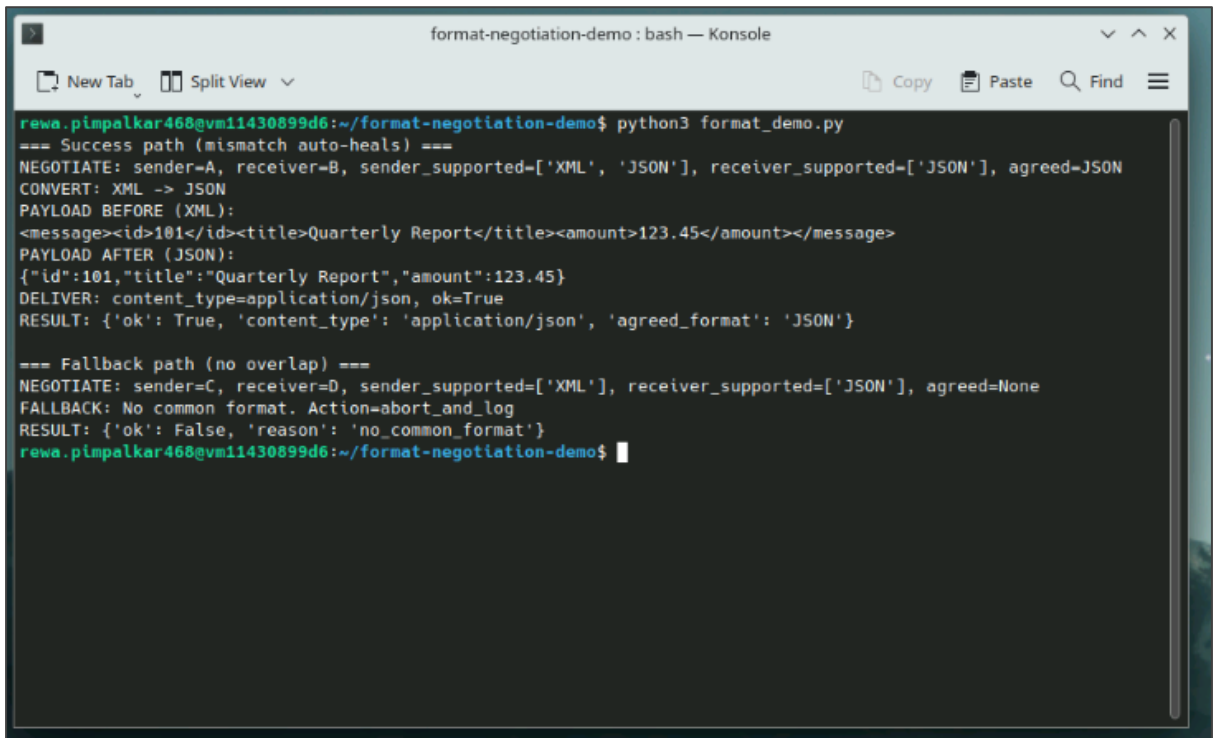
def xml_bytes_to_dict(b: bytes) -> Dict:
    root = ET.fromstring(b.decode("utf-8"))
    out = {}
    for child in root:
        text = child.text or ""
        try:
            num = float(text)
            out[child.tag] = int(num) if num.is_integer() else num
        except:
            out[child.tag] = text
    return out

def convert_bytes(src_fmt: str, dst_fmt: str, payload: bytes) -> bytes:

```

1.7 Run the **format_demo.py** file using the following command:

python3 format_demo.py



```
format-negotiation-demo: bash — Konsole
New Tab Split View Copy Paste Find
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$ python3 format_demo.py
=== Success path (mismatch auto-heals) ===
NEGOTIATE: sender=A, receiver=B, sender_supported=['XML', 'JSON'], receiver_supported=['JSON'], agreed=JSON
CONVERT: XML -> JSON
PAYLOAD BEFORE (XML):
<message><id>101</id><title>Quarterly Report</title><amount>123.45</amount></message>
PAYLOAD AFTER (JSON):
{"id":101,"title":"Quarterly Report","amount":123.45}
DELIVER: content_type=application/json, ok=True
RESULT: {'ok': True, 'content_type': 'application/json', 'agreed_format': 'JSON'}

=== Fallback path (no overlap) ===
NEGOTIATE: sender=C, receiver=D, sender_supported=['XML'], receiver_supported=['JSON'], agreed=None
FALLBACK: No common format. Action=abort_and_log
RESULT: {'ok': False, 'reason': 'no_common_format'}
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$
```

Note: This scenario negotiates JSON, auto-converts XML → JSON, and delivers successfully; the second scenario finds no shared format and triggers a safe FALLBACK.

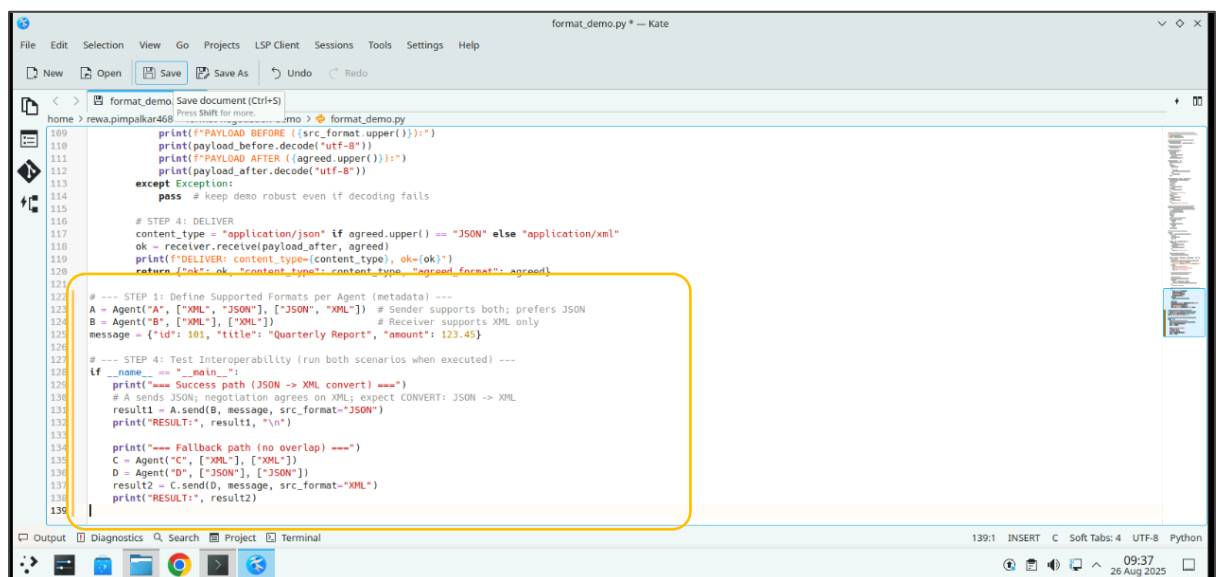
Step 2: Try scenario variations in the format negotiation code

2.1 Edit the agent definitions at the bottom of **format_demo.py** with the following code to test JSON → XML conversion, then save the file:

```
# --- STEP 1: Define Supported Formats per Agent (metadata) ---
A = Agent("A", ["XML", "JSON"], ["JSON", "XML"]) # Sender supports both; prefers JSON
B = Agent("B", ["XML"], ["XML"])                # Receiver supports XML only
message = {"id": 101, "title": "Quarterly Report", "amount": 123.45}

# --- STEP 4: Test Interoperability (run both scenarios when executed) ---
if __name__ == "__main__":
    print("=== Success path (JSON -> XML convert) ===")
    # A sends JSON; negotiation agrees on XML; expect CONVERT: JSON -> XML
    result1 = A.send(B, message, src_format="JSON")
    print("RESULT:", result1, "\n")

    print("=== Fallback path (no overlap) ===")
    C = Agent("C", ["XML"], ["XML"])
    D = Agent("D", ["JSON"], ["JSON"])
    result2 = C.send(D, message, src_format="XML")
    print("RESULT:", result2)
```



```
format_demo.py — Kate
File Edit Selection View Go Projects LSP Client Sessions Tools Settings Help
New Open Save Save As Undo Redo
format_demo.py
109 print(f"PAYLOAD BEFORE ({src_format.upper()}):")
110 print(payload_before.decode("utf-8"))
111 print(f"PAYLOAD AFTER ({agreed.upper()}):")
112 print(payload_after.decode("utf-8"))
113 except Exception:
114     pass # keep demo robust even if decoding fails
115
116 # STEP 4: DELIVER
117 content_type = "application/json" if agreed.upper() == "JSON" else "application/xml"
118 ok = receiver.receive(payload_after, agreed)
119 print(f"DELIVER: content_type={content_type}, ok={ok}")
120 return f"ok={ok}, src_format={content_type}, agreed_format={agreed}"
121
122 # --- STEP 1: Define Supported Formats per Agent (metadata) ---
123 A = Agent("A", ["XML", "JSON"], ["JSON", "XML"]) # Sender supports both; prefers JSON
124 B = Agent("B", ["XML"], ["XML"])                # Receiver supports XML only
125 message = {"id": 101, "title": "Quarterly Report", "amount": 123.45}
126
127 # --- STEP 4: Test Interoperability (run both scenarios when executed) ---
128 if __name__ == "__main__":
129     print("=== Success path (JSON -> XML convert) ===")
130     # A sends JSON; negotiation agrees on XML; expect CONVERT: JSON -> XML
131     result1 = A.send(B, message, src_format="JSON")
132     print("RESULT:", result1, "\n")
133
134     print("=== Fallback path (no overlap) ===")
135     C = Agent("C", ["XML"], ["XML"])
136     D = Agent("D", ["JSON"], ["JSON"])
137     result2 = C.send(D, message, src_format="XML")
138     print("RESULT:", result2)
```

2.2 Rerun the code using the command given below:

python3 format_demo.py

```
format-negotiation-demo: bash — Konsole
New Tab Split View
Copy Paste Find
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$ python3 format_demo.py
=== Success path (JSON -> XML convert) ===
NEGOTIATE: sender=A, receiver=B, sender_supported=['XML', 'JSON'], receiver_supported=['XML'], agreed=XML
CONVERT: JSON -> XML
PAYLOAD BEFORE (JSON):
{"id":101,"title":"Quarterly Report","amount":123.45}
PAYLOAD AFTER (XML):
<message><id>101</id><title>Quarterly Report</title><amount>123.45</amount></message>
DELIVER: content_type=application/xml, ok=True
RESULT: {'ok': True, 'content_type': 'application/xml', 'agreed_format': 'XML'}

=== Fallback path (no overlap) ===
NEGOTIATE: sender=C, receiver=D, sender_supported=['XML'], receiver_supported=['JSON'], agreed=None
FALLBACK: No common format. Action=abort_and_log
RESULT: {'ok': False, 'reason': 'no_common_format'}
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$
```

Note: The Success path: agents negotiate **XML**, auto-convert the sender's **JSON → XML**, and deliver successfully; then a **no-overlap** scenario triggers a safe **FALLBACK**.

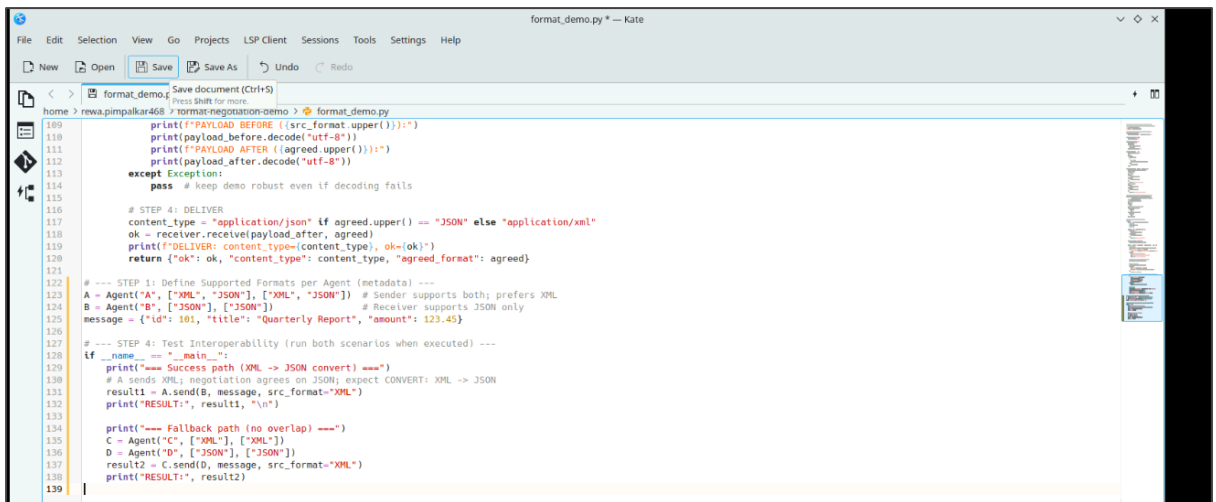
- 2.3 Edit the code at the bottom of **format_demo.py** with the following code to test XML → JSON conversion, then save the file:

```
# --- STEP 1: Define Supported Formats per Agent (metadata) ---
A = Agent("A", ["XML", "JSON"], ["XML", "JSON"]) # Sender supports both; prefers XML
B = Agent("B", ["JSON"], ["JSON"]) # Receiver supports JSON only
message = {"id": 101, "title": "Quarterly Report", "amount": 123.45}

# --- STEP 4: Test Interoperability (run both scenarios when executed) ---
if __name__ == "__main__":
    print("=== Success path (XML -> JSON convert) ===")
    # A sends XML; negotiation agrees on JSON; expect CONVERT: XML -> JSON
    result1 = A.send(B, message, src_format="XML")
    print("RESULT:", result1, "\n")

    print("=== Fallback path (no overlap) ===")
    C = Agent("C", ["XML"], ["XML"])
    D = Agent("D", ["JSON"], ["JSON"])
    result2 = C.send(D, message, src_format="XML")
```

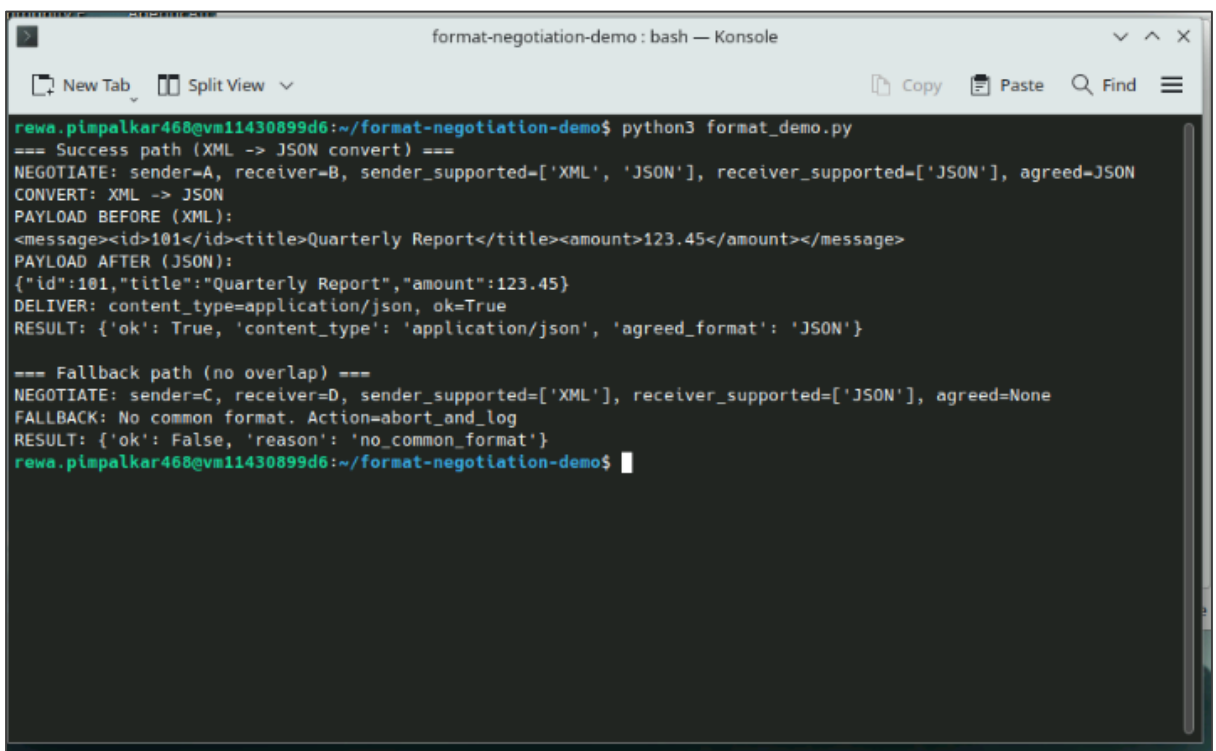
```
print("RESULT:", result2)
```



```
format_demo.py
109 print(f"PAYLOAD BEFORE ({src_format.upper()}):")
110 print(payload_before.decode("utf-8"))
111 print(f"PAYLOAD AFTER ({agreed.upper()}):")
112 print(payload_after.decode("utf-8"))
113 except Exception:
114     pass # keep demo robust even if decoding fails
115
116 # STEP 4: DELIVER
117 content_type = "application/json" if agreed.upper() == "JSON" else "application/xml"
118 ok = receiver.receive(payload_after, agreed)
119 print(f"DELIVER: content_type={content_type}, ok={ok}")
120 return {"ok": ok, "content_type": content_type, "agreed_format": agreed}
121
122 # --- STEP 1: Define Supported Formats per Agent (metadata) ---
123 A = Agent("A", ["XML", "JSON"], ["XML", "JSON"]) # Sender supports both; prefers XML
124 B = Agent("B", ["JSON"], ["JSON"]) # Receiver supports JSON only
125 message = {"id": 101, "title": "Quarterly Report", "amount": 123.45}
126
127 # --- STEP 4: Test Interoperability (run both scenarios when executed) ---
128 if __name__ == "__main__":
129     print("=== Success path (XML -> JSON convert) ===")
130     # A sends XML; negotiation agrees on JSON; expect CONVERT: XML -> JSON
131     result1 = A.send(B, message, src_format="XML")
132     print("RESULT:", result1, "\n")
133
134     print("=== Fallback path (no overlap) ===")
135     C = Agent("C", ["XML"], ["XML"])
136     D = Agent("D", ["JSON"], ["JSON"])
137     result2 = C.send(D, message, src_format="XML")
138     print("RESULT:", result2)
```

2.4 Rerun the code using the command given below:

```
python3 format_demo.py
```



```
format-negotiation-demo: bash — Konsole
New Tab Split View Copy Paste Find
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$ python3 format_demo.py
=== Success path (XML -> JSON convert) ===
NEGOTIATE: sender=A, receiver=B, sender_supported=['XML', 'JSON'], receiver_supported=['JSON'], agreed=JSON
CONVERT: XML -> JSON
PAYLOAD BEFORE (XML):
<message><id>101</id><title>Quarterly Report</title><amount>123.45</amount></message>
PAYLOAD AFTER (JSON):
{"id":101,"title":"Quarterly Report","amount":123.45}
DELIVER: content_type=application/json, ok=True
RESULT: {'ok': True, 'content_type': 'application/json', 'agreed_format': 'JSON'}

=== Fallback path (no overlap) ===
NEGOTIATE: sender=C, receiver=D, sender_supported=['XML'], receiver_supported=['JSON'], agreed=None
FALLBACK: No common format. Action=abort_and_log
RESULT: {'ok': False, 'reason': 'no_common_format'}
rewa.pimpalkar468@vm11430899d6:~/format-negotiation-demo$
```

Note: The Success case: agents agree on **JSON**, auto-convert the sender's **XML** → **JSON**, and deliver; then the **no-overlap** scenario aborts safely with a logged **FALLBACK**.

By following these steps, you have successfully demonstrated response-format negotiation end-to-end: agents declare capabilities, agree on a common format, auto-convert when required, and fail safely with clear logs.